



Fullstack Web Development

Session 8 | Week 8/Day 2 | 120 min

Laravel - Part02

Instructor: Ms. Liev Nova



090667393



Content

- **About Laravel**
- Composer
- Laravel Installation
- Artisan
- Configuration
- Directory Structure
- Frontend

About Laravel

- What is Laravel?
 - **Laravel** is a powerful and modern **PHP web application framework** used to build dynamic and scalable web applications.
 - A web framework provides:
 - A ready-made project structure
 - Built-in tools and libraries
 - Faster and cleaner development process
 - Better security and maintainability
 - Laravel allows developers to focus on building application features instead of writing repetitive code.

About Laravel

- **Key Features of Laravel**
 - Elegant and simple syntax
 - MVC (Model-View-Controller) architecture
 - Built-in authentication system
 - Database migration and ORM (Eloquent)
 - Routing system
 - Task scheduling and queues
 - API development support
 - Unit and integration testing
 - Security protection

About Laravel

- **Why Laravel?**
 - **Progressive Framework** – Laravel is beginner-friendly and easy to learn.
 - It provides:
 - Clear documentation
 - Tutorials and examples
 - Large learning community
 - Video courses and guides
 - Developers can start small and gradually build advanced applications.

About Laravel

- **Why Laravel?**
 - **Scalable Framework** – Laravel supports scalable application development.
 - Features include:
 - Redis caching support
 - Queue management
 - Session handling
 - Horizontal scaling
 - Performance optimization
 - Laravel can power:
 - Small websites, Business systems
 - Enterprise applications, Large-scale APIs

About Laravel

- **Why Laravel?**
 - **Community Framework** – Laravel has a huge global community.
 - Benefits:
 - Thousands of reusable packages
 - Open-source contributions
 - Frequent updates
 - Community support
 - Developer-friendly ecosystem
 - Many developers worldwide contribute to improving Laravel.

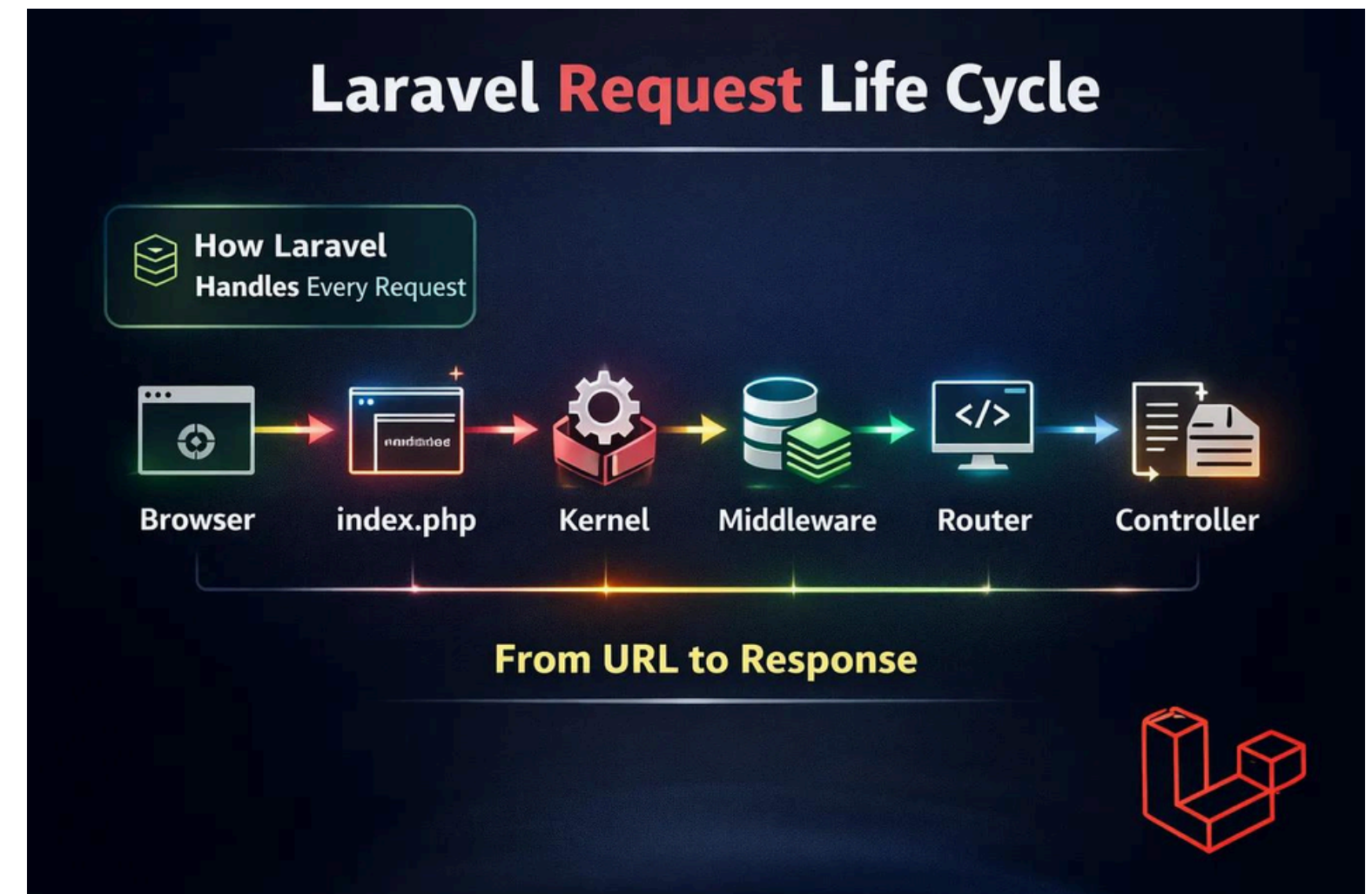
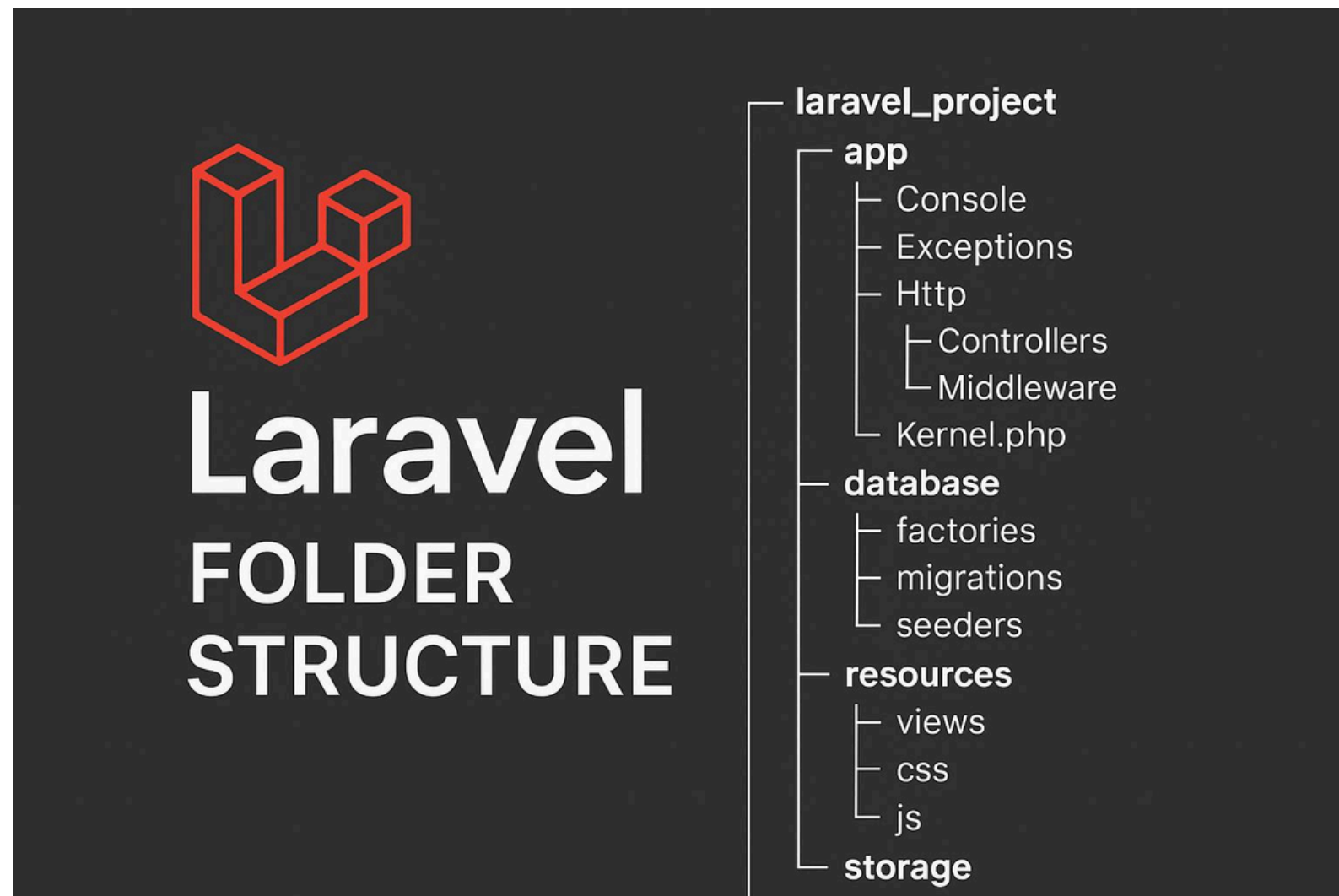
About Laravel

- Popular Laravel Features

Feature	Description
Routing	Manage application URLs
Blade Template	Laravel templating engine
Eloquent ORM	Database interaction using models
Middleware	Filter HTTP requests
Migration	Database version control
Artisan	Command-line tool
Authentication	Login & user management
API Support	RESTful API development

About Laravel

- **Laravel Architecture**
 - Laravel follows the MVC architecture:



About Laravel

- **Laravel Architecture Pattern (MVC)**

- MVC stands for:

- **M → Model** – Handles database logic

- **V → View** – User interface

- **C → Controller** – Business logic and request handling

- This architecture helps developers organize code clearly and efficiently.

About Laravel

- **Laravel Architecture Pattern (MVC)**
 - **Model (M)** – The **Model** represents the **data layer** of the application.
 - It is responsible for:
 - Database interaction
 - Business logic
 - Data validation
 - Managing application data
 - Laravel Model Location – **app/Models**

About Laravel

- **Laravel Architecture Pattern (MVC)**
 - **Model (M)** – The **Model** represents the **data layer** of the application.
 - Example: User Model
 - The User model interacts with the users table in the database.
 - Example operations:
 - Insert data
 - Update data
 - Delete data
 - Retrieve data

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    //
}
```

About Laravel

- **Laravel Architecture Pattern (MVC)**
 - **View (V)** – The **View** represents the **presentation layer**.
 - It controls:
 - User interface (UI)
 - Displaying data
 - HTML pages
 - Blade templates
 - Laravel View Location – `app/Models`

About Laravel

- **Laravel Architecture Pattern (MVC)**
 - **View (V)** – The **View** represents the **presentation layer**.
 - Example – `welcome.blade.php`
 - HTML
 - CSS
 - JavaScript
 - Blade Template Engine

```
<h1>Welcome to Laravel</h1>
```

About Laravel

- **Laravel Architecture Pattern (MVC)**
 - **Controller (C)** – The **Controller** acts as the **middleman** between:
 - Model
 - View
 - It handles:
 - User requests
 - Application logic
 - Processing data
 - Returning responses
 - Laravel View Location – `app/Http/Controllers`

About Laravel

- **Laravel Architecture Pattern (MVC)**

- **Controller (C)** – The **Controller** acts as the **middleman** between:

- Example – UserController

- The controller:

- Receives request
 - Uses Model to get data
 - Sends data to View
 - Returns response to user

```
<?php

namespace App\Http\Controllers;

use App\Models\User;

class UserController extends Controller
{
    public function index()
    {
        $users = User::all();

        return view('users', compact('users'));
    }
}
```

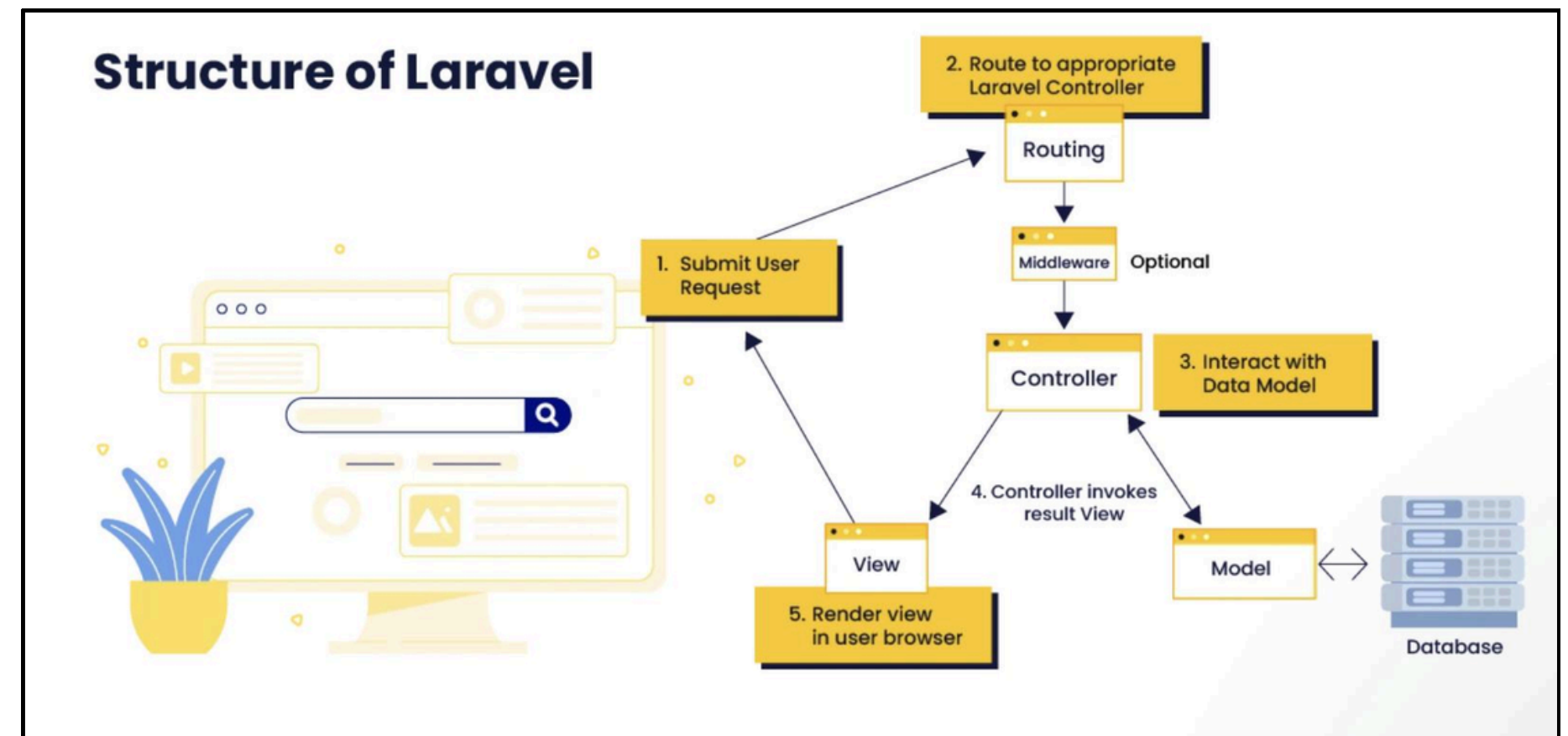
About Laravel

- **Laravel Architecture Pattern (MVC)**
 - Real Example

Component	Example
Model	User.php
View	users.blade.php
Controller	UserController.php

About Laravel

- **Laravel Architecture Pattern (MVC)**
 - **Laravel MVC Flow**
 - User sends request
 - Route receives request
 - Controller handles request
 - Model interacts with database
 - View displays data
 - Response returned to user



About Laravel

- **Server Requirements**

- PHP $\geq$ 8.0
 - Ctype PHP Extension
 - cURL PHP Extension
 - DOM PHP Extension
 - Fileinfo PHP Extension
 - Filter PHP Extension
 - Hash PHP Extension
 - Mbstring PHP Extension
 - OpenSSL PHP Extension
 - PCRE PHP Extension
 - PDO PHP Extension
 - Session PHP Extension
 - Tokenizer PHP Extension
 - XML PHP Extension
- Command to check php extension: `php -m`

Content

- About Laravel
- **Composer**
- Laravel Installation
- Artisan
- Configuration
- Directory Structure
- Frontend

Composer

- **What is Composer?**

- Composer is a dependency management tool for PHP.
- It allows developers to:
 - Manage PHP libraries and packages
 - Install project dependencies automatically
 - Update packages easily
 - Maintain consistent project environments
- Composer is widely used in modern PHP frameworks such as:
 - Laravel
 - Symfony
 - CodeIgniter

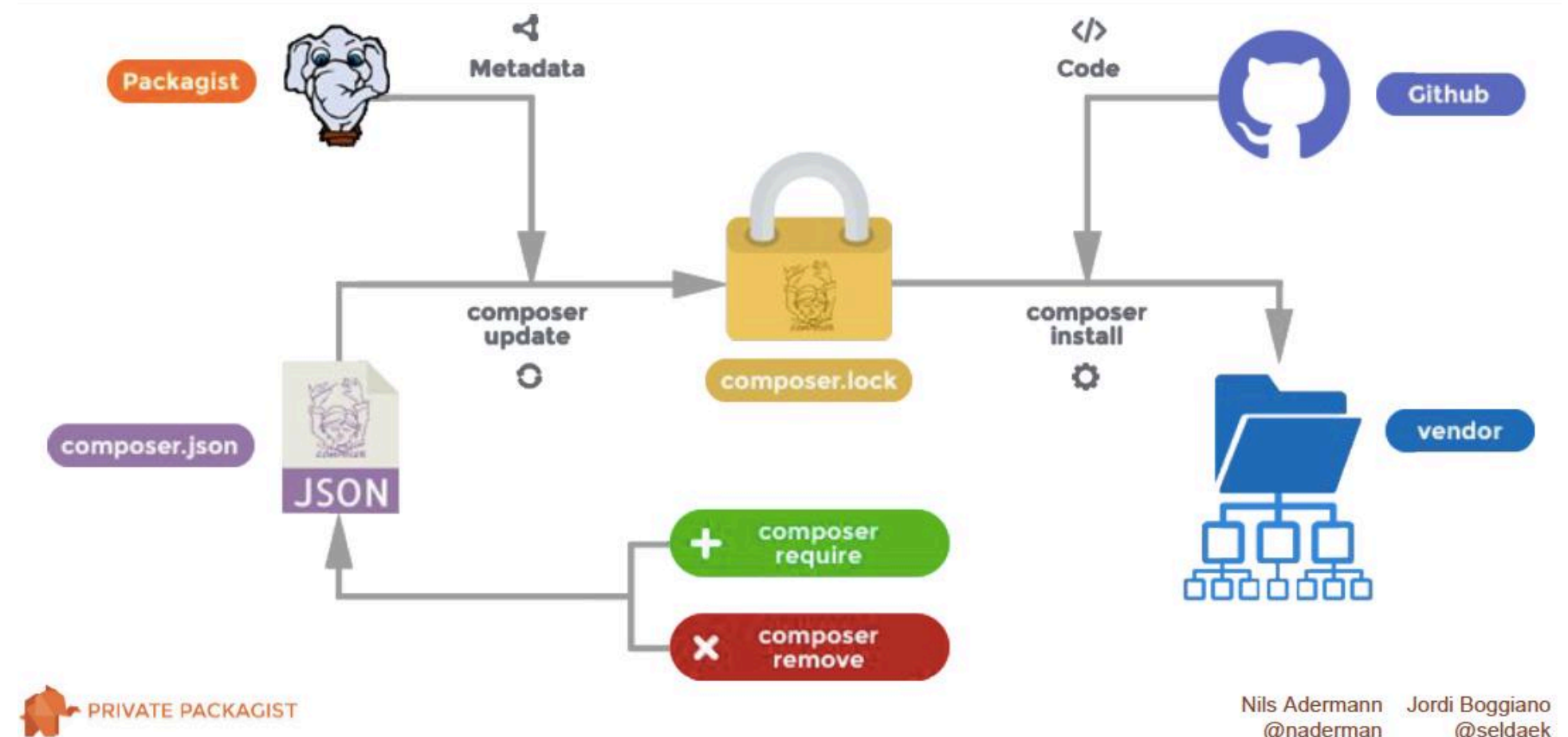
Composer

- **Why Use Composer?**
 - Without Composer:
 - Developers manually download libraries
 - Dependency conflicts may occur
 - Updating packages is difficult
 - With Composer:
 - Automatic dependency management
 - Easy package installation
 - Version control for libraries
 - One-command updates
 - Faster development workflow

Composer

- **How Composer Works**

- Your project depends on:
 - Library A
 - Library B
- Library A also depends on:
 - Library C
 - Library D
- Composer automatically:
 - Detects required libraries
 - Finds compatible versions
 - Downloads packages
 - Installs dependencies
 - Updates packages when needed



Composer

- Important Composer Files

File	Purpose
composer.json	Stores project dependencies
composer.lock	Stores exact installed versions
vendor/	Contains installed packages

- Example: `composer.json`

```
{
  "require": {
    "laravel/framework": "^12.0"
  }
}
```

Composer

- **Composer Commands**

Command	Description
composer install	Install dependencies
composer update	Update dependencies
composer require package-name	Install new package
composer remove package-name	Remove package
composer --version	Check Composer version

- **Example commands:**

- Install Dependencies – *composer install*
- Install Laravel – *composer create-project laravel/laravel YourProjectName*

**System Requirements: composer latest v2.5.3 requires php7.2.5 +*

Composer

- **Composer Installation – Linux / Unix / macOS**

- Step 1 – Download the installer to the current directory

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
```

- Step2 – Verify the installer SHA-384

```
php -r "if (hash_file('sha384', 'composer-setup.php') ===  
'55ce33d7678c5a611085589f1f3ddf8b3c52d662cd01d4ba75c0ee0459970c2200a51f492d557530c71c15d8dba01eae') { echo  
'Installer verified'; } else { echo 'Installer corrupt'; unlink('composer-setup.php'); } echo PHP_EOL;"
```

- Step 3 – Install Composer

```
php composer-setup.php --filename=composer  
sudo mv composer /usr/local/bin #only for mac
```

- Remove installer

```
php -r "unlink('composer-setup.php');"
```

Composer

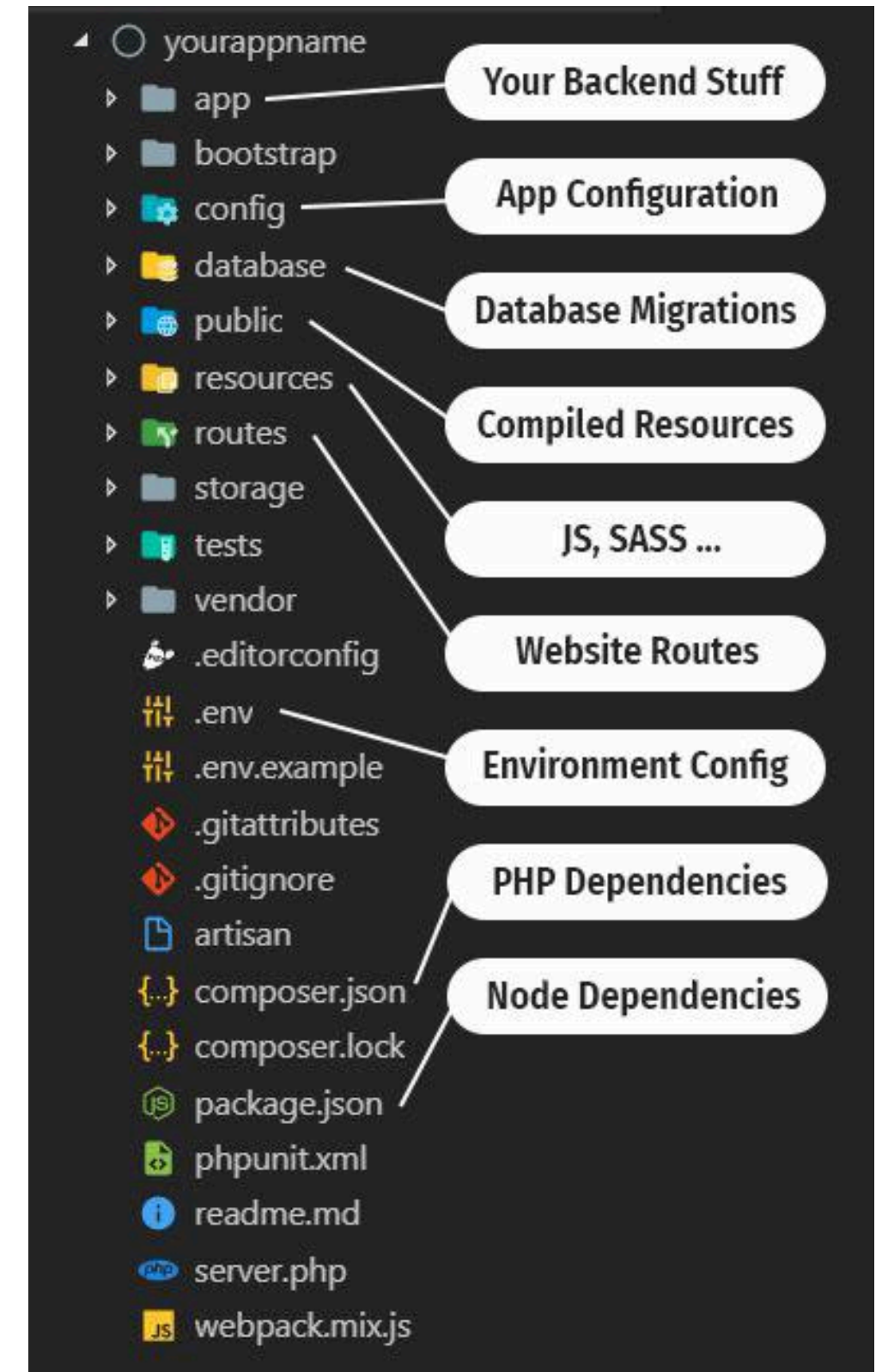
- **Composer Installation – Windows Installation**

- Step 1 – Download Composer Installer:

<https://getcomposer.org/Composer-Setup.exe>

- Step 2 – Run the installer and follow setup instructions.

- Step 3 – Verify installation: *composer --version*



Content

- About Laravel
- Composer
- **Laravel Installation**
- Artisan
- Configuration
- Directory Structure
- Frontend

Laravel Installation

- **Requirements**

- Before installing Laravel, make sure your computer has:
 - PHP
 - Composer
 - Zip extension
 - Fileinfo extension
 - Terminal / Command Prompt

Laravel Installation

- **Create New Laravel Project**
 - **Method 1:** Using Composer

```
composer create-project laravel/laravel example-app
```

- This command will create a new Laravel project named: *example-app*
- **Install Specific Laravel Version**
 - Example: Install Laravel version 9.5.2

```
composer create-project laravel/laravel="9.5.2" example-app --prefer-dist
```

- Use this when your project or lesson requires a specific Laravel version.

Laravel Installation

- **Create New Laravel Project**
 - **Method 2:** Using Laravel Installer
 - Install Laravel Installer Globally and Create New Laravel Project

```
composer global require laravel/installer  
laravel new example-app
```

Laravel Installation

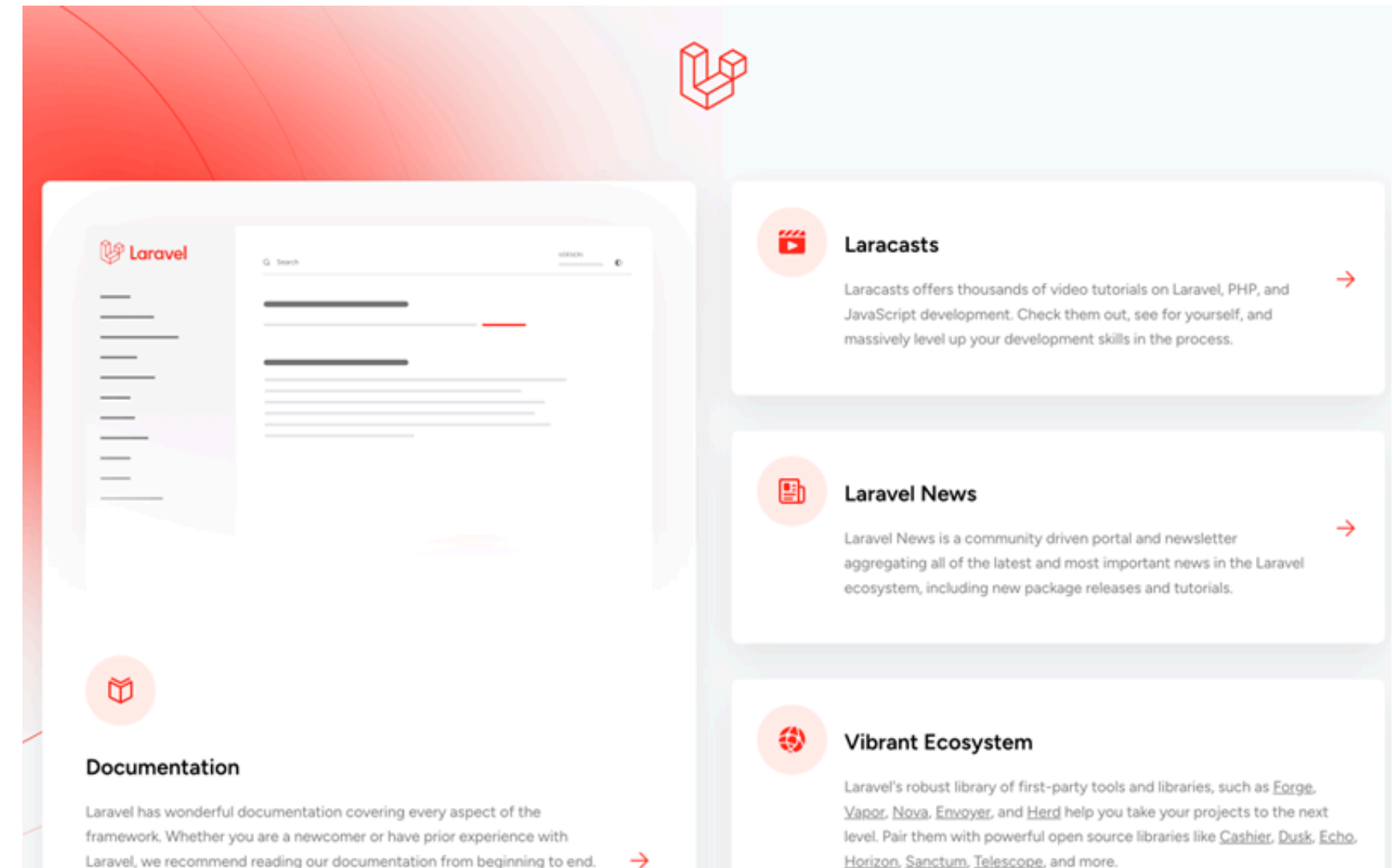
- **Run Laravel Project**

- After the project is created, go into the project folder then start Laravel local server:

```
cd example-app  
php artisan serve
```

- Laravel will run at:

```
http://127.0.0.1:8000
```



Laravel Installation

- **Some erros during installation**

```
Failed to download laravel/laravel from dist: The zip extension and  
unzip/7z commands are both missing, skipping. The php.ini used by your  
command-line PHP is: C:\php\php.ini
```

- This error happens because:

- PHP zip extension is disabled
 - unzip/7z is not installed
 - PHP configuration is missing required extensions

- **Solution**

```
* Window: open the php.ini file and uncomment  
;extension=zip to extension=zip  
;extension=fileinfo to extension=fileinfo  
* Linux  
sudo apt install zip unzip php-zip
```

- In case composer is very slow

```
composer config --global repo.packagist composer
```

Content

- About Laravel
- Composer
- Laravel Installation
- **Artisan**
- Configuration
- Directory Structure
- Frontend

Artisan Console

- **What is Artisan?**

- Laravel includes a powerful command-line tool called Artisan.
- Artisan helps developers:
 - Create files quickly
 - Run Laravel commands
 - Manage database migrations
 - Generate controllers and models
 - Clear cache
 - Test applications
 - Interact with Laravel using terminal
- Artisan is located in the root directory of every Laravel project.

Artisan Console

- **Artisan Command Structure**

- Basic syntax: `php artisan command-name`

- Example: `php artisan serve`

- **View All Artisan Commands**

- Use the list command: `php artisan list`
 - This displays all available Artisan commands.

- Every command also includes a "help" screen which displays and describes the command's available arguments and options

`php artisan help migrate`

or

`php artisan migrate -h`

Artisan Console

- **Common Artisan Commands**

Command	Description
php artisan serve	Start development server
php artisan migrate	Run database migrations
php artisan make:model	Create model
php artisan make:controller	Create controller
php artisan make:migration	Create migration
php artisan route:list	Show all routes
php artisan cache:clear	Clear cache
php artisan config:clear	Clear configuration cache

Artisan Console

- **Example Commands**
 - Create Controller
 - *php artisan make:controller UserController*
 - Create Model
 - *php artisan make:model User*
 - Run Migration
 - *php artisan migrate*

Artisan Tinker

- **What is Tinker?**

- Tinker is Laravel's interactive command-line environment.
- It allows you to:
 - Execute PHP code
 - Test Laravel features
 - Work with database models
 - Debug application logic
 - Test helper functions

- **Start Tinker** `php artisan tinker`

```
>>> doc view
>>> doc str::kebab
>>> show view
>>> show str::kebab
```

```
php artisan tinker
>>> 7*5
>>> 7>5
>>> 7=="7"
>>> 7==="7"
>>> strlen("Hello World")
>>> view('welcome')
>>> view('welcome')->render()
>>> Str::kebab('HelloWorld')
>>> str("HelloWorld")->kebab()
>>> Str::snake("HelloWorld")
>>> help
```

Content

- About Laravel
- Composer
- Laravel Installation
- Artisan
- **Configuration**
- Directory Structure
- Frontend

Configuration

- **Laravel Configuration**
 - Laravel stores all configuration files inside the: *config/* directory.
 - These files allow developers to configure:
 - Database connection
 - Mail server
 - Cache system
 - Queue settings
 - Application timezone
 - Encryption key
 - Session settings
 - File storage

Configuration

- **Common Configuration Files**

File	Purpose
app.php	Application settings
database.php	Database configuration
mail.php	Mail server configuration
cache.php	Cache settings
queue.php	Queue configuration
filesystems.php	File storage configuration

Configuration

- **Application Overview Command**

- Laravel provides the about command to display application information.
- Show Application Information

```
php artisan about
```

- This command displays:

- Laravel version
- PHP version
- Database
- Environment
- Cache driver
- Queue driver
- Server information

Configuration

- **Application Overview Command**
 - Laravel provides the about command to display application information.
 - Show Environment Information Only

```
php artisan about --only=environment
```

Configuration

- **Environment Configuration**

- **The `.env` File**

- Laravel uses an environment file called: `.env`
 - This file stores environment-specific configuration values.

- **`.env.example` File**

- Laravel includes a template file: `.env.example`
 - During installation: `.env.example` is copied to `.env`

Configuration

- **Environment Configuration**

- **Purpose of .env**

- The **.env** file stores:

- Database credentials
 - Application URL
 - Mail configuration
 - API keys
 - Application environment
 - Debug mode

- Example: **.env** file

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:xxxxxx
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=
```

Configuration

- **Environment Types**

Environment	Purpose
local	Development environment
production	Live server
testing	Automated testing

- **Access Environment Variables**

- Laravel uses the `env()` function.
- *Example: `env('APP_NAME')`*

- **Configuration Example**

- Inside config files:
 - `'name' => env('APP_NAME', 'Laravel'),`
 - Read value from `.env`
 - If not found, use default value `Laravel`

Configuration

- **Important Security Rule**
 - Never Commit **.env** File
 - The .env file should NOT be uploaded to:
 - GitHub
 - GitLab
 - Source control repositories
 - Because it contains sensitive information such as:
 - Database passwords
 - API keys
 - Secret tokens

Configuration

- **Laravel Environment Encryption**
 - Laravel supports encrypted environment files.
 - Benefits:
 - Safer production deployment
 - Secure source control storage
 - Protect sensitive credentials
- **Useful Environment Commands**
 - Clear Config Cache
 - `php artisan config:clear`
 - Cache Configurations
 - `php artisan config:cache`

Content

- About Laravel
- Composer
- Laravel Installation
- Artisan
- Configuration
- **Directory Structure**
- Frontend

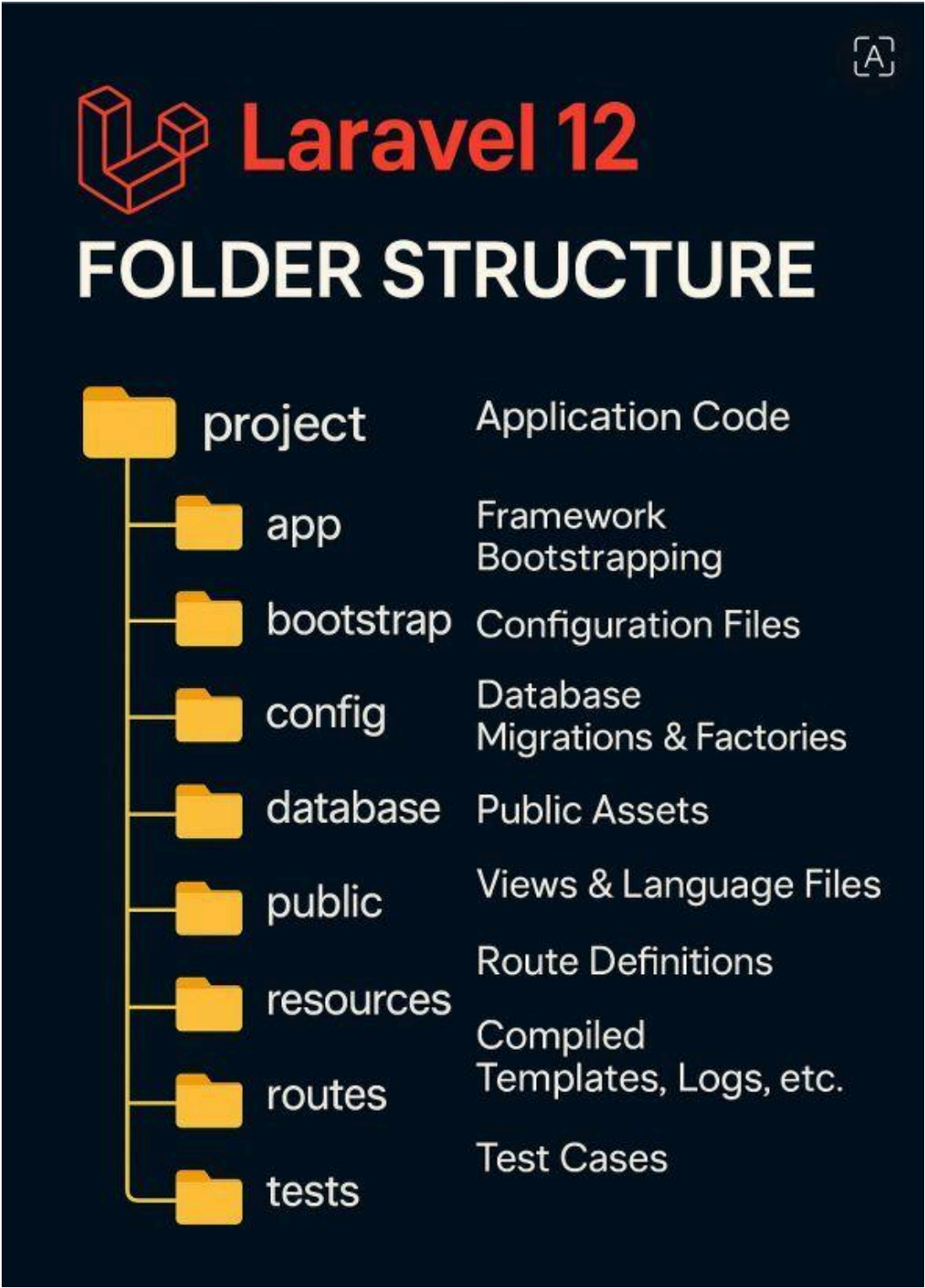
Directory Structure

- **Laravel Directory Structure**

- Laravel provides a clean and organized project structure for both:
 - Small applications
 - Large enterprise systems
- Laravel's default structure helps developers:
 - Organize code clearly
 - Separate application responsibilities
 - Improve maintainability
 - Scale projects more easily

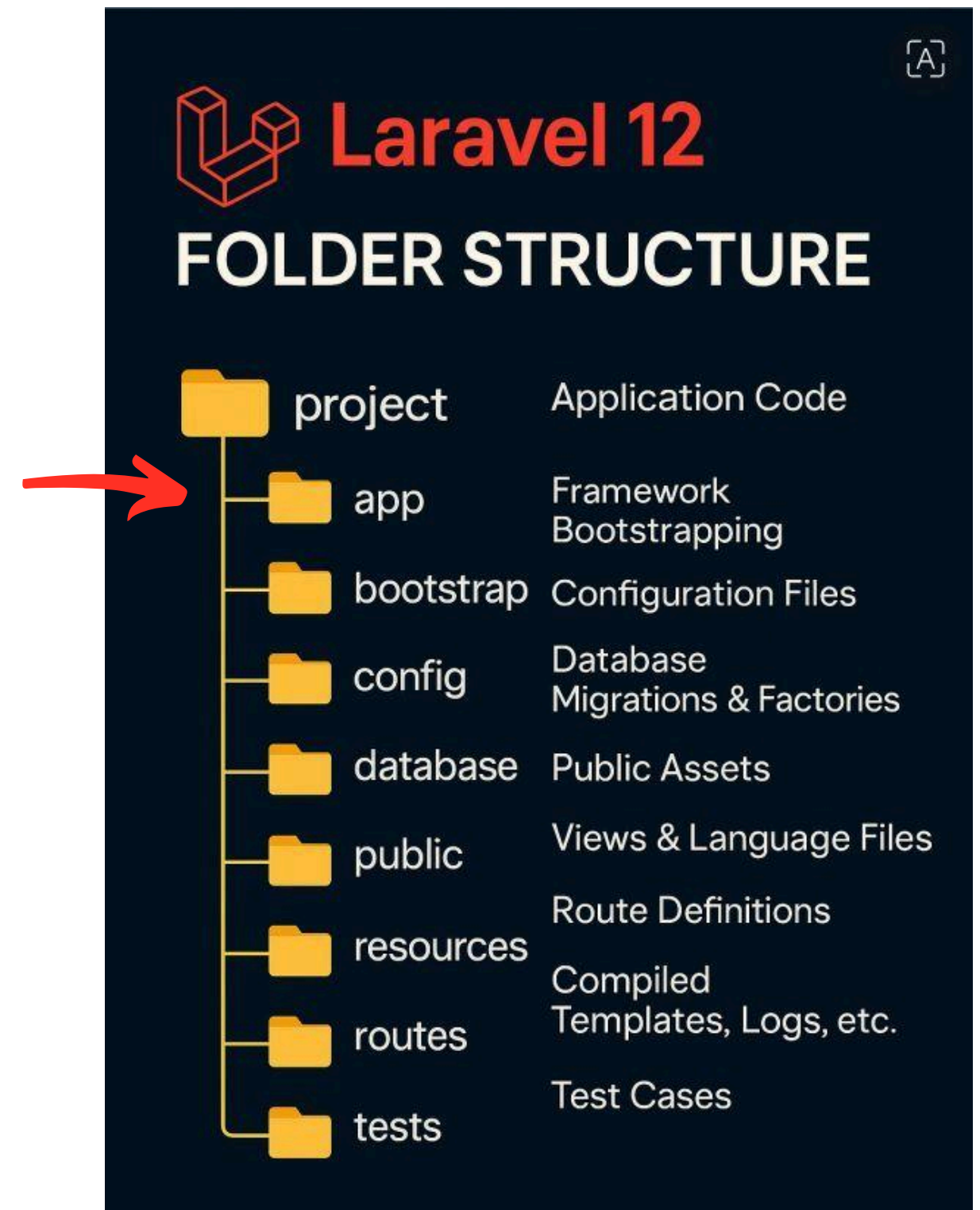
Directory Structure

Directory	Purpose
app	Core application code
bootstrap	Framework bootstrapping
config	Configuration files
database	Migrations and seeders
lang	Language files
public	Public entry point
resources	Views and frontend assets
routes	Route definitions
storage	Logs and cache
tests	Automated tests
vendor	Composer dependencies



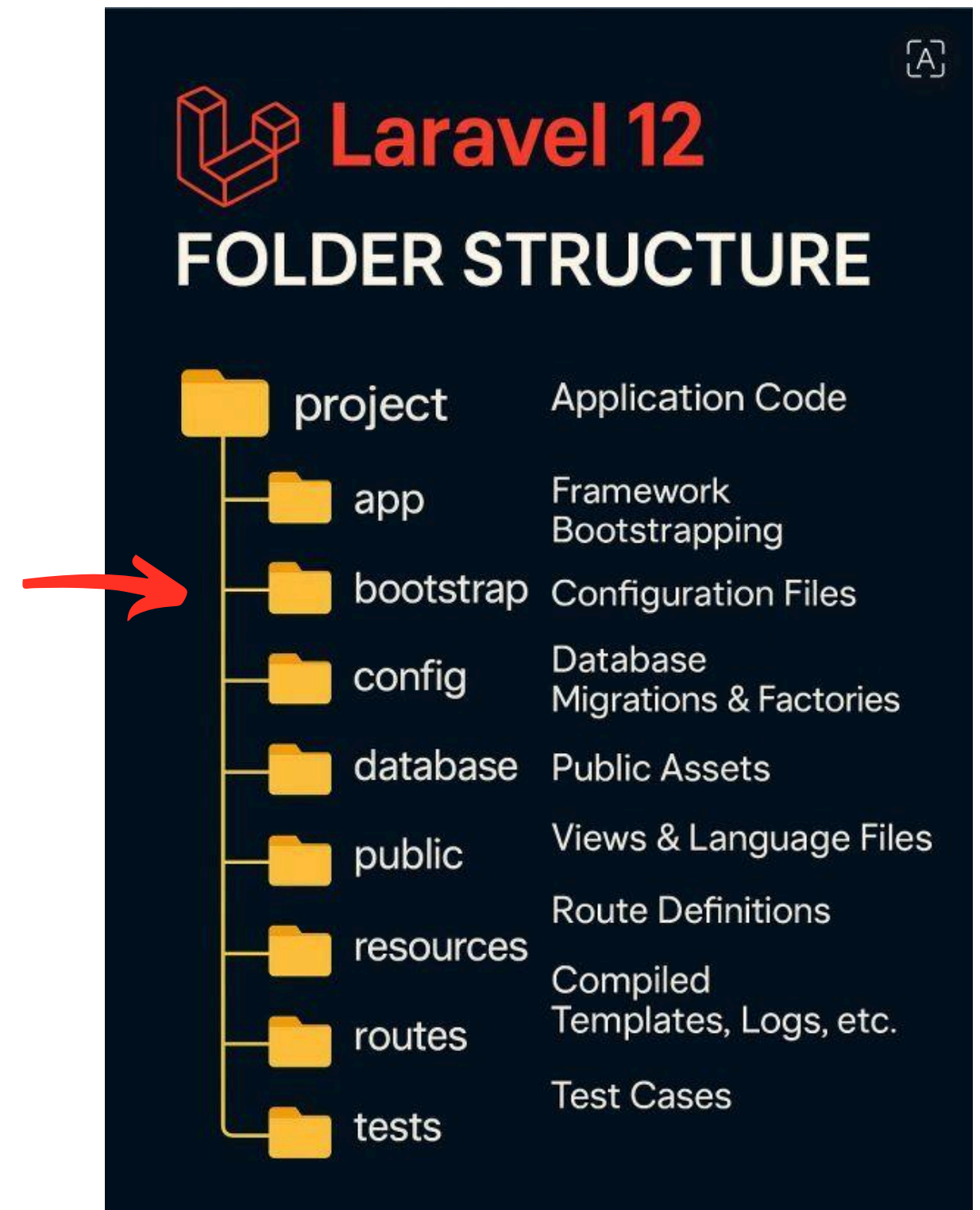
Directory Structure

- The **app** Directory
 - Location: **app/**
 - Contains the core code of your application.
 - Most classes are stored here:
 - Controllers
 - Models
 - Middleware
 - Jobs
 - Events
 - Notifications



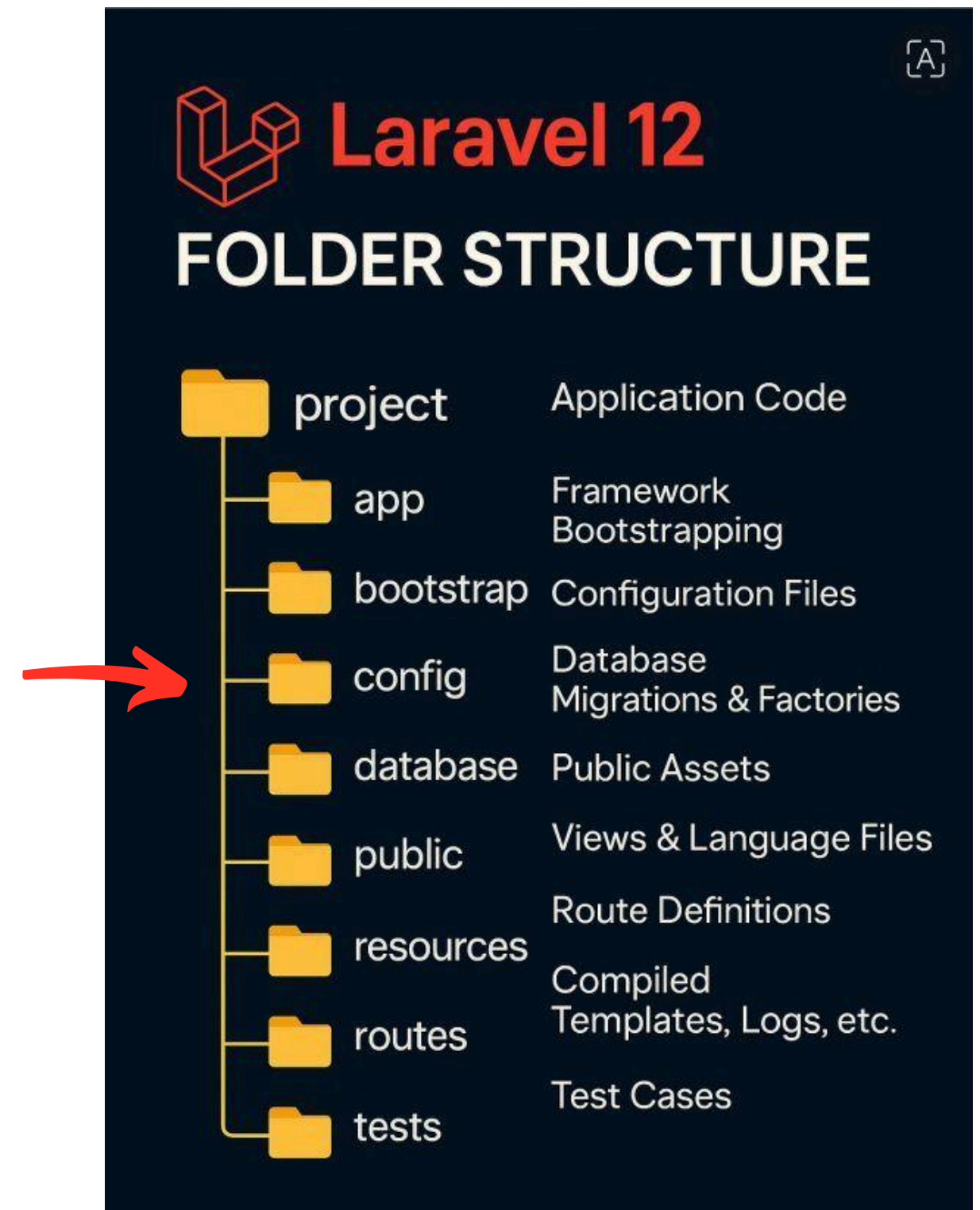
Directory Structure

- The **bootstrap** Directory
 - Location: **bootstrap/**
 - Contains:
 - **app.php**
 - Framework bootstrap files
 - Cache files
 - Used for:
 - Framework initialization
 - Performance optimization
 - Usually developers do NOT modify these files.



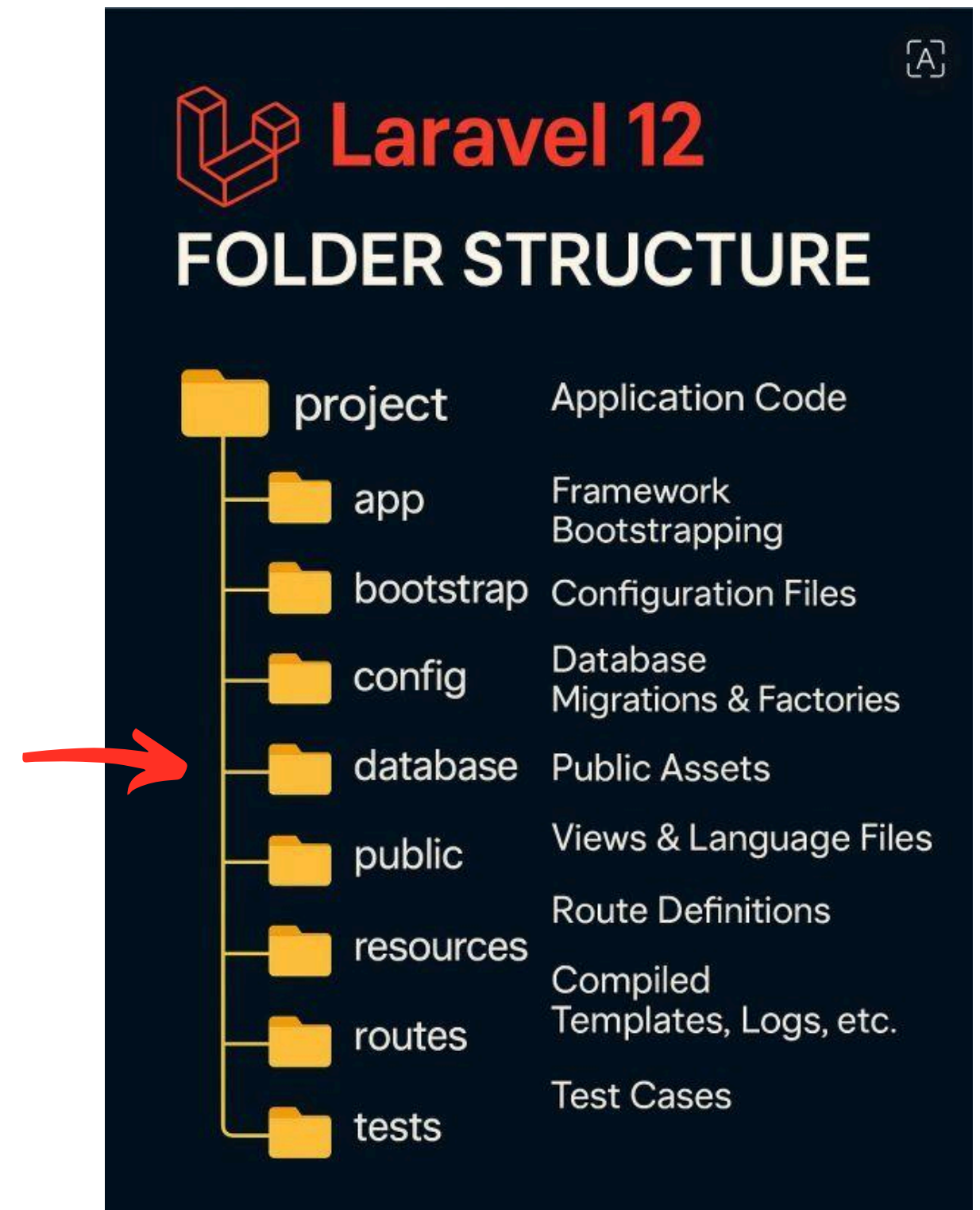
Directory Structure

- The **config** Directory
 - Location: **config/**
 - Contains all configuration files:
 - Database settings
 - Mail configuration
 - Cache configuration
 - configuration
 - App settings



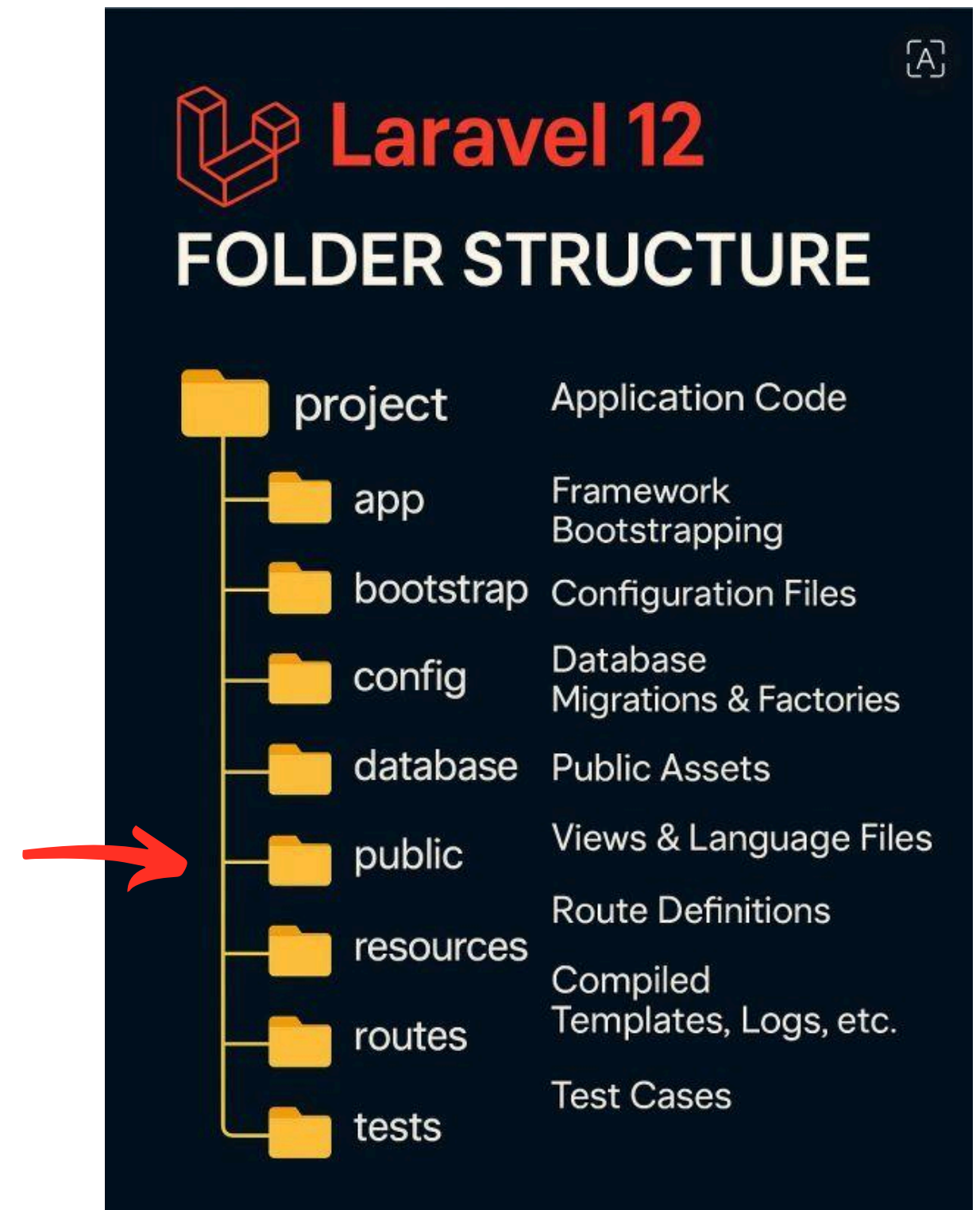
Directory Structure

- The **database** Directory
 - Location: **database/**
 - Contains:
 - Migrations
 - Seeders
 - Factories
 - Used for database structure and test data.



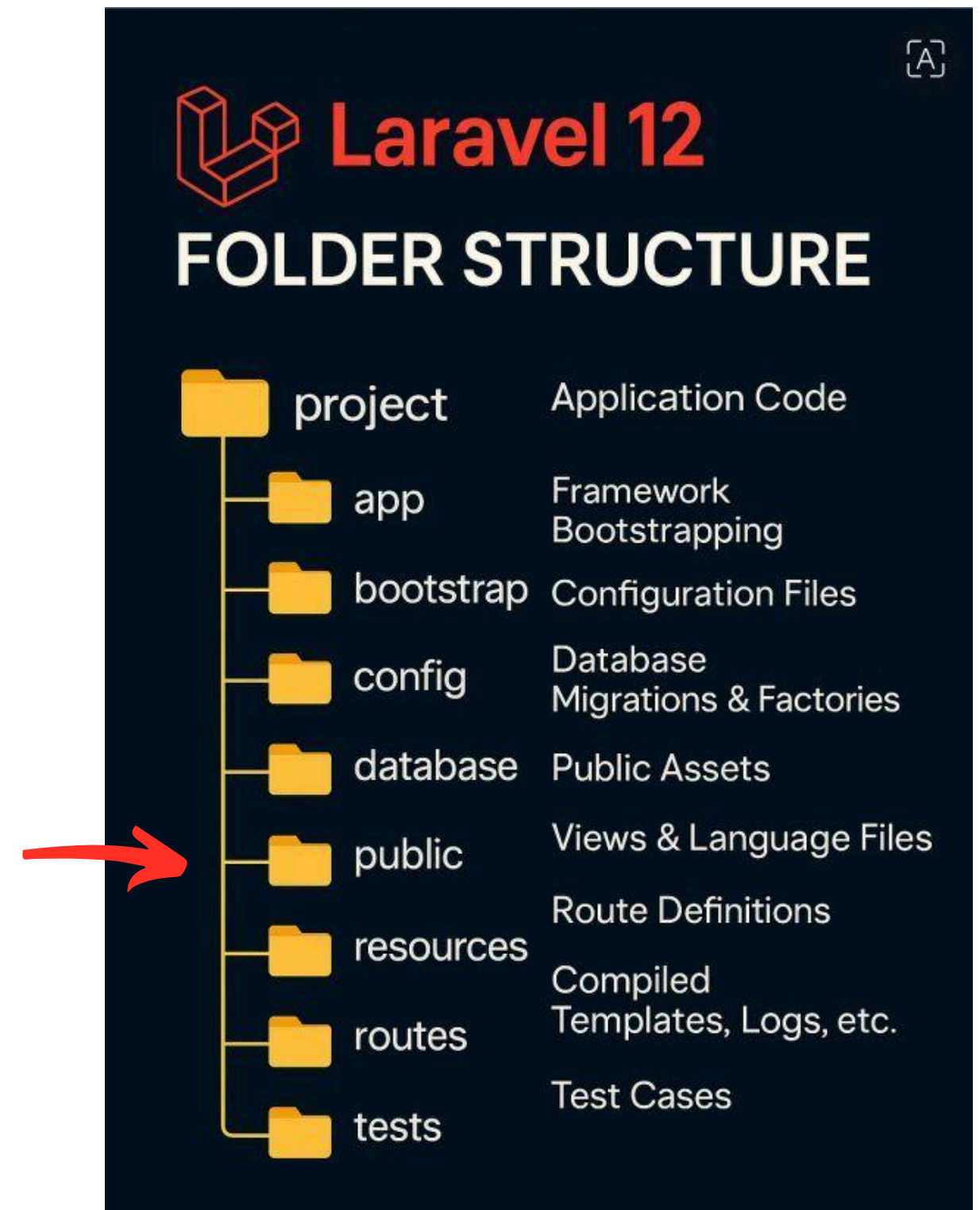
Directory Structure

- The **lang** Directory
 - Location: **lang/**
 - Contains language translation files.
 - Used for:
 - Multi-language applications
 - Localization



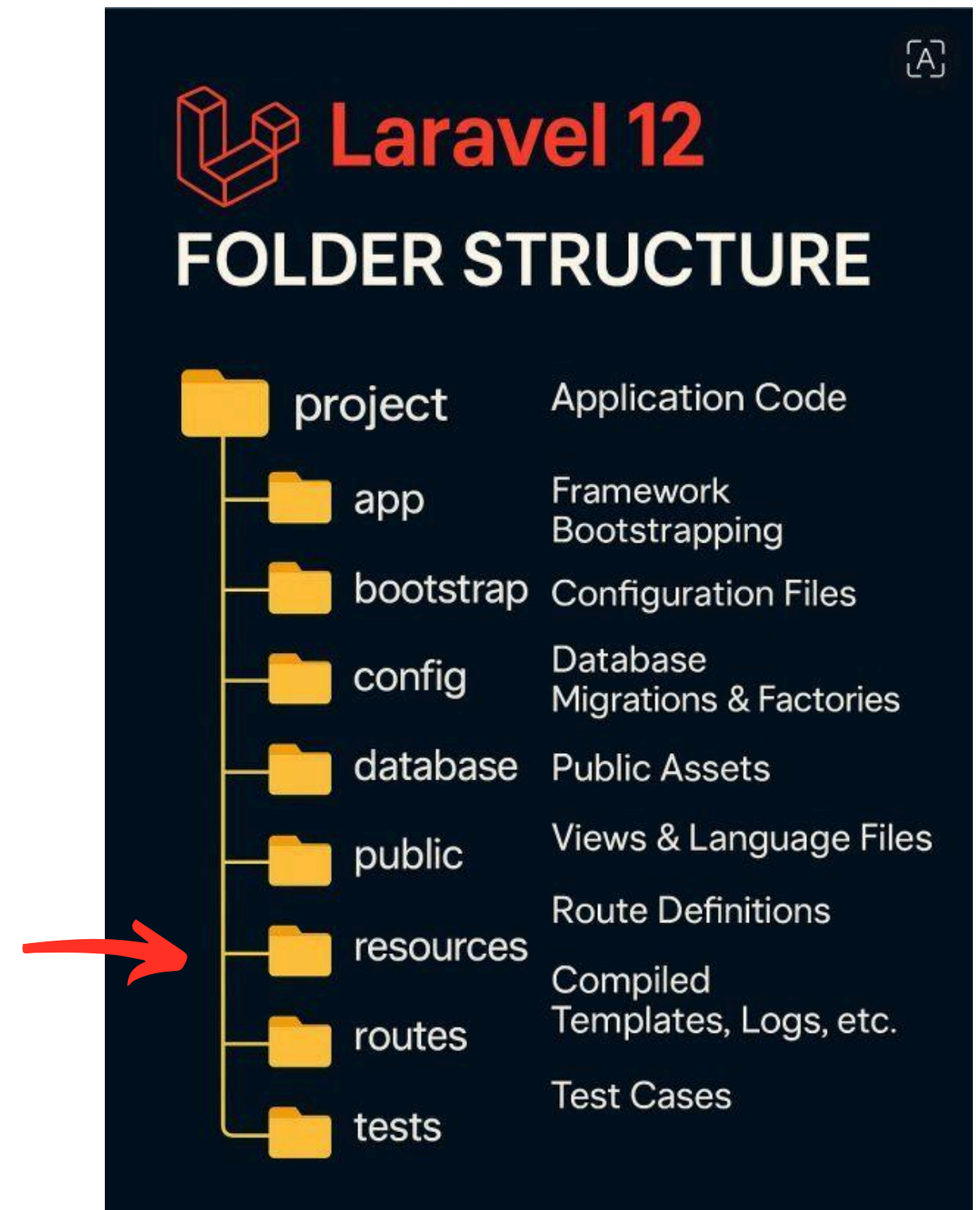
Directory Structure

- The **public** Directory
 - Location: **public/**
 - Contains:
 - **index.php**
 - Images
 - CSS
 - JavaScript
 - Public assets
 - This is the main entry point of the application.



Directory Structure

- The **resources** Directory
 - Location: **resources/**
 - Contains:
 - Blade templates
 - Raw CSS
 - JavaScript files
 - Frontend resources



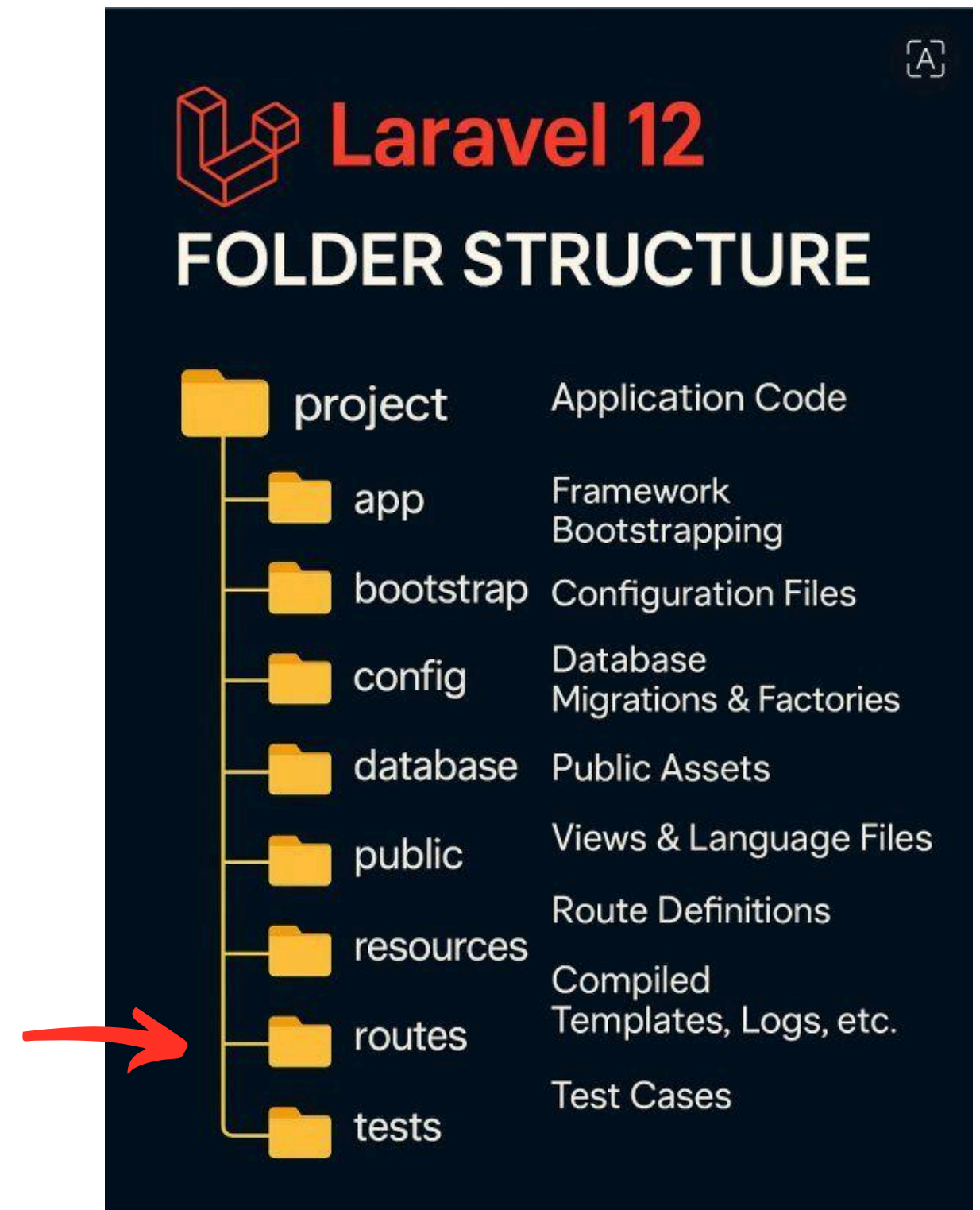
Directory Structure

- The **routes** Directory

- Location: **routes/**
- Contains all route definitions.

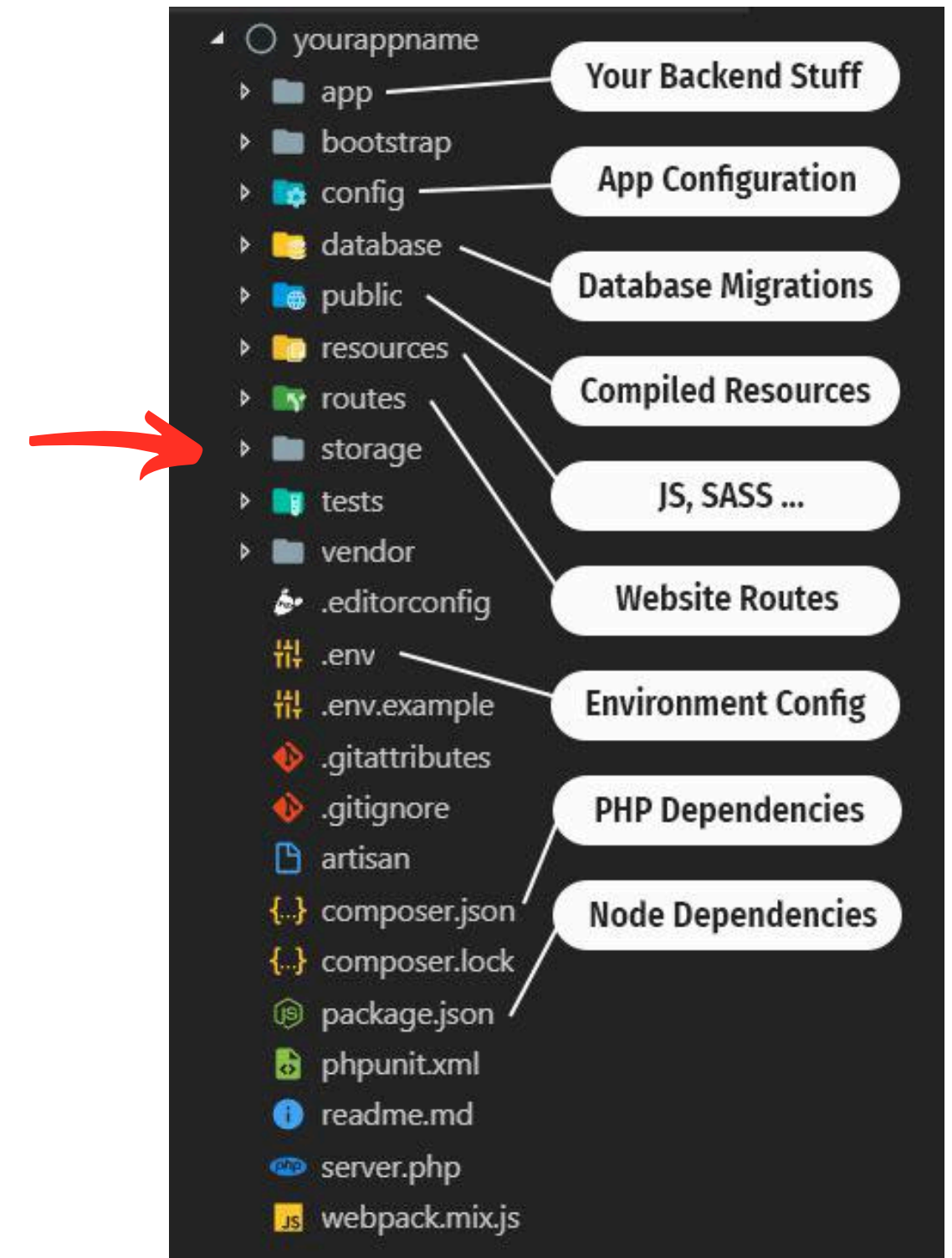
- Important route files:

File	Purpose
web.php	Web routes
api.php	API routes
console.php	Console commands
channels.php	Broadcasting channels



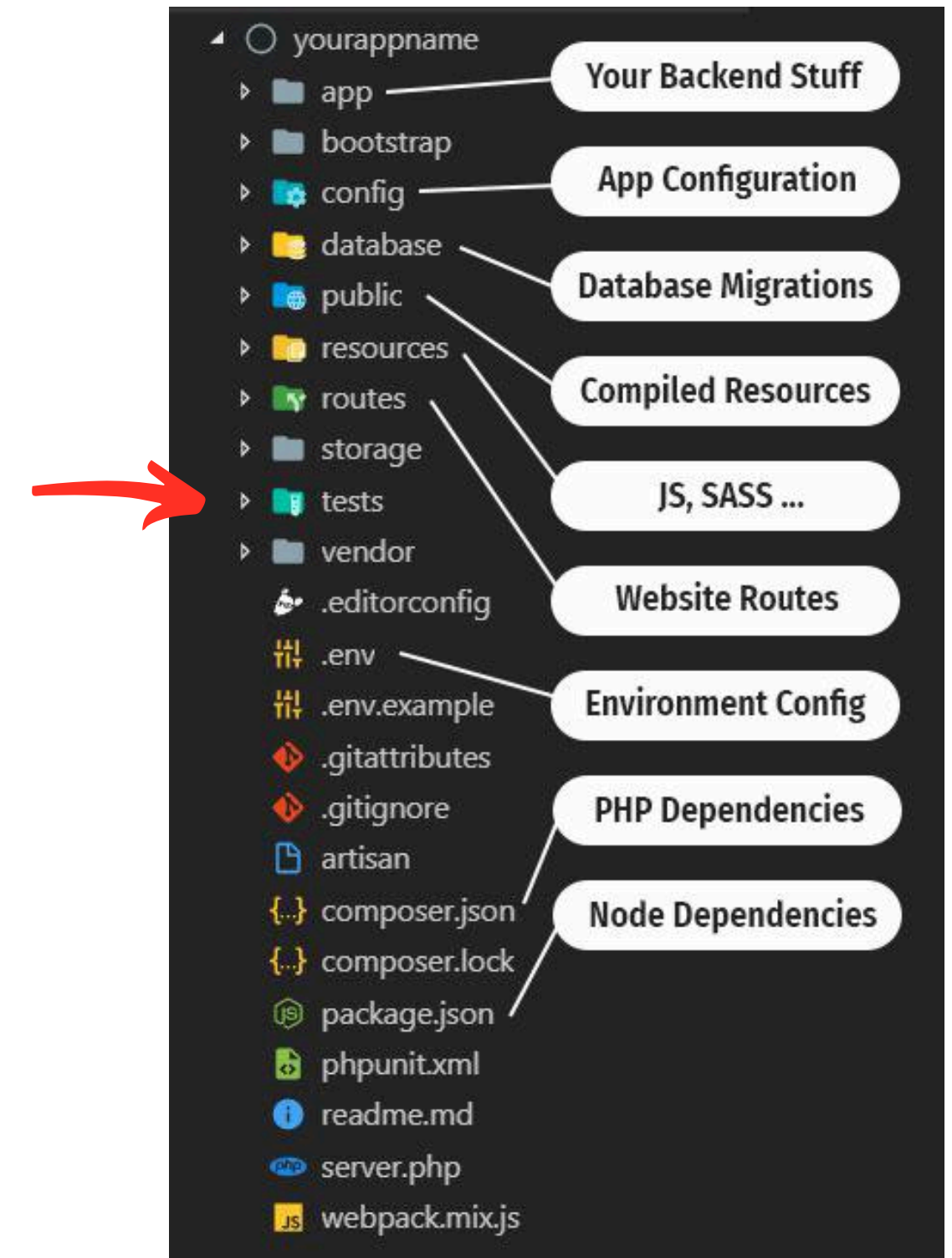
Directory Structure

- The **storage** Directory
 - Location: **storage/**
 - Stores:
 - Logs
 - Cache
 - Sessions
 - Uploaded files
 - Compiled Blade templates



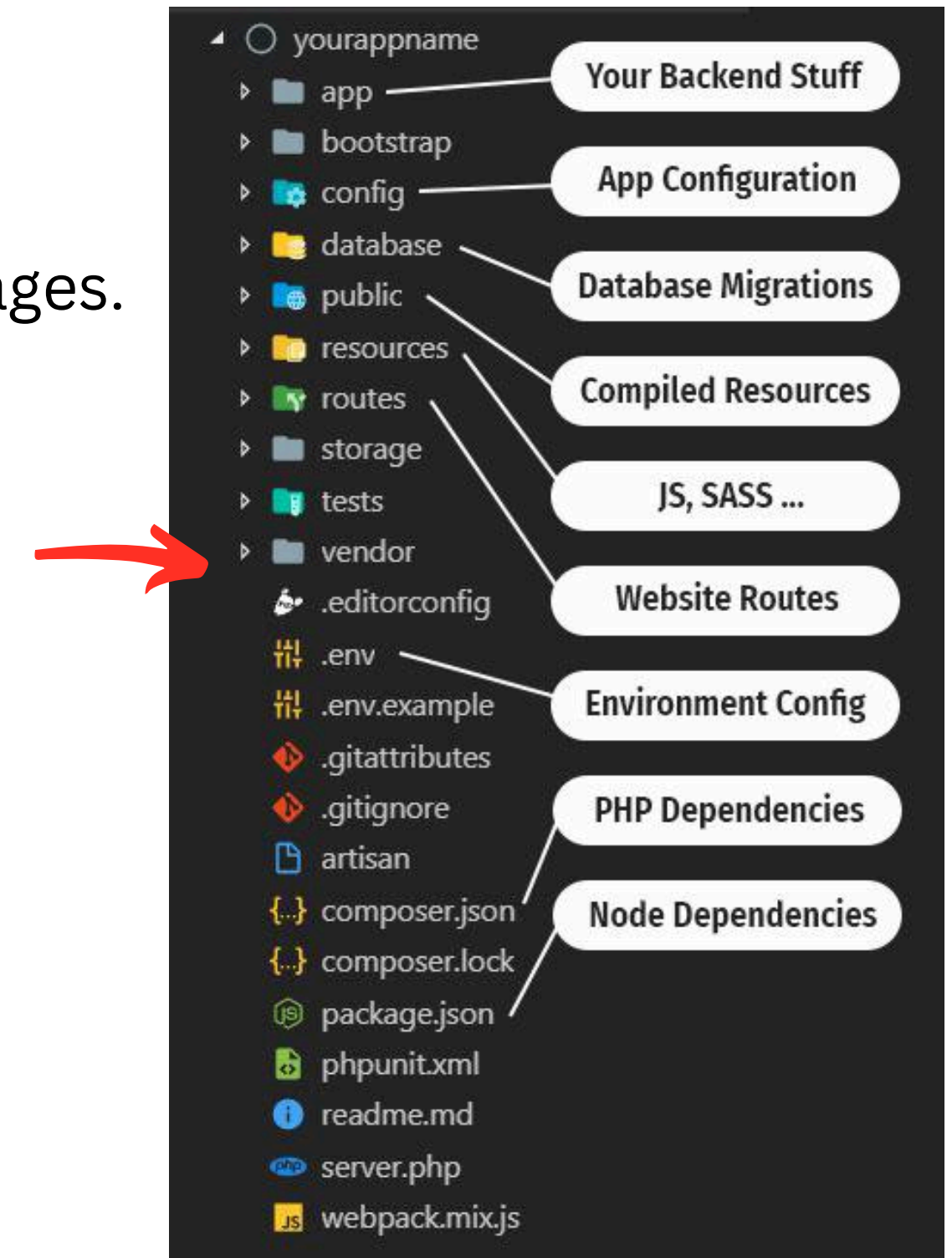
Directory Structure

- The **tests** Directory
 - Location: **tests/**
 - Contains automated tests.
 - Used for:
 - Unit testing
 - Feature testing
 - Laravel uses PHPUnit for testing.



Directory Structure

- The **vendor** Directory
 - Location: **vendor/**
 - Contains Composer dependencies and third-party packages.
 - Automatically generated by: **composer install**



Content

- About Laravel
- Composer
- Laravel Installation
- Artisan
- Configuration
- Directory Structure
- **Frontend**

Frontend

- **Laravel Frontend Development**

- Laravel provides multiple ways to build frontend applications.
- Laravel supports:
 - Blade Templates
 - Livewire
 - Vue.js
 - React
 - Inertia.js
- This gives developers flexibility to create:
 - Simple websites
 - Dynamic dashboards
 - Single Page Applications (SPA)
 - Modern web applications

Frontend

- **Blade Template Engine**

- What is Blade?

- Blade is Laravel's built-in templating engine.

- It is:

- Lightweight
 - Simple
 - Fast
 - Easy to use

- Blade provides clean syntax for:

- Displaying data
 - Loops
 - Conditions
 - Layouts
 - Components

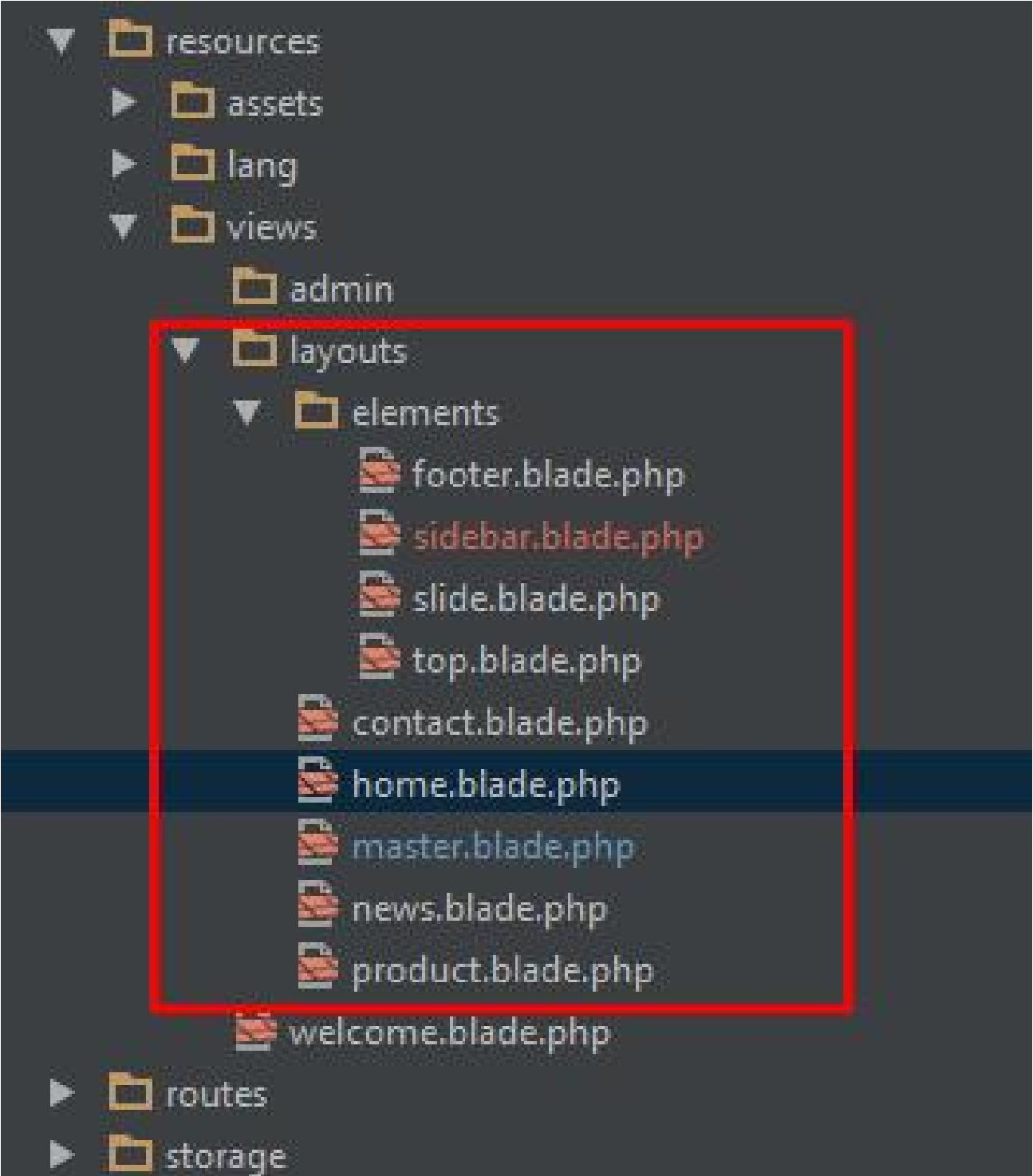
- **Example**

```
<div>
    @foreach ($users as $user)
        Hello, {{ $user->name }} <br />
    @endforeach
</div>
```

Frontend

- Blade Features

Feature	Description
{{ }}	Display data
@foreach	Loop through data
@if	Conditional statements
@include	Include views
@extends	Template inheritance
@section	Define sections



Frontend

- **Laravel Livewire**
 - What is Livewire?
 - Laravel Livewire is a framework for building dynamic Laravel frontends without writing much JavaScript.
 - It allows developers to create:
 - Interactive interfaces
 - Real-time updates
 - Modern UI behavior
 - using mostly:
 - PHP
 - Blade

Frontend

- **Livewire Features**

- Dynamic components
- Real-time interaction
- Minimal JavaScript
- Laravel integration
- Faster development

- **Livewire Example Use Cases**

- Search systems
- Real-time forms
- Dynamic tables
- Notifications
- Dashboards

Frontend

- **Using Vue / React with Laravel**
 - Laravel also supports modern JavaScript frameworks such as:
 - Vue.js
 - React
 - These frameworks help build:
 - Modern UI
 - Single Page Applications (SPA)
 - Interactive dashboards
 - Mobile-like web experiences

Frontend

- **Problem Without Additional Tooling**
 - Using Laravel with Vue or React directly can create challenges such as:
 - Client-side routing
 - Authentication handling
 - Data hydration
 - API communication
 - State management

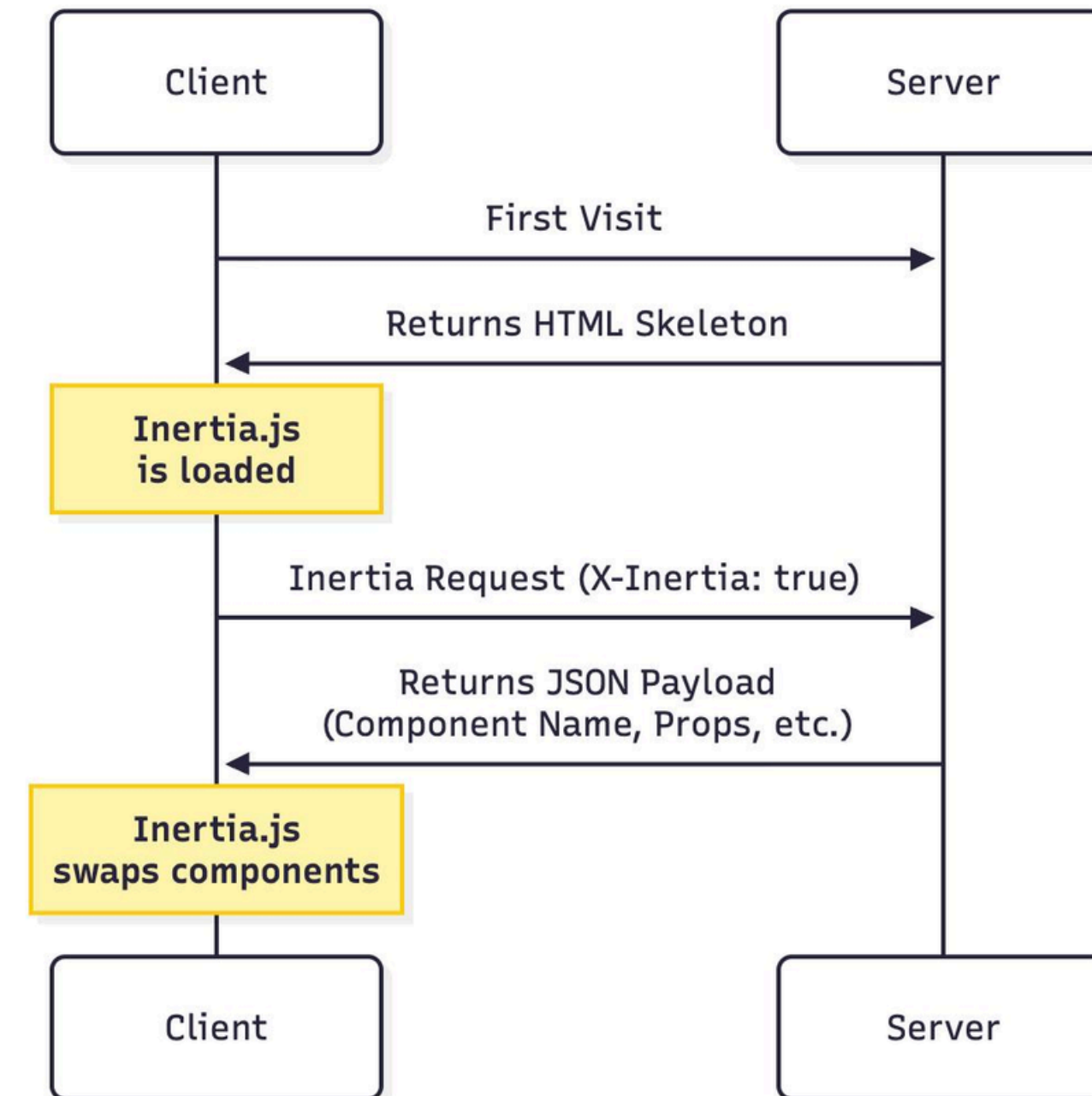
Frontend

- **Inertia.js**

- What is Inertia?
 - Inertia.js bridges Laravel with:
 - Vue
 - React
 - It allows developers to build modern frontend applications while still using:
 - Laravel routes
 - Laravel controllers
 - Laravel authentication
- inside a single project.

Frontend

- **Inertia.js**
 - Inertia Benefits
 - No separate API needed
 - Full-stack development
 - Single code repository
 - Modern frontend experience
 - Easy Laravel integration





THANK YOU

For your attention !

